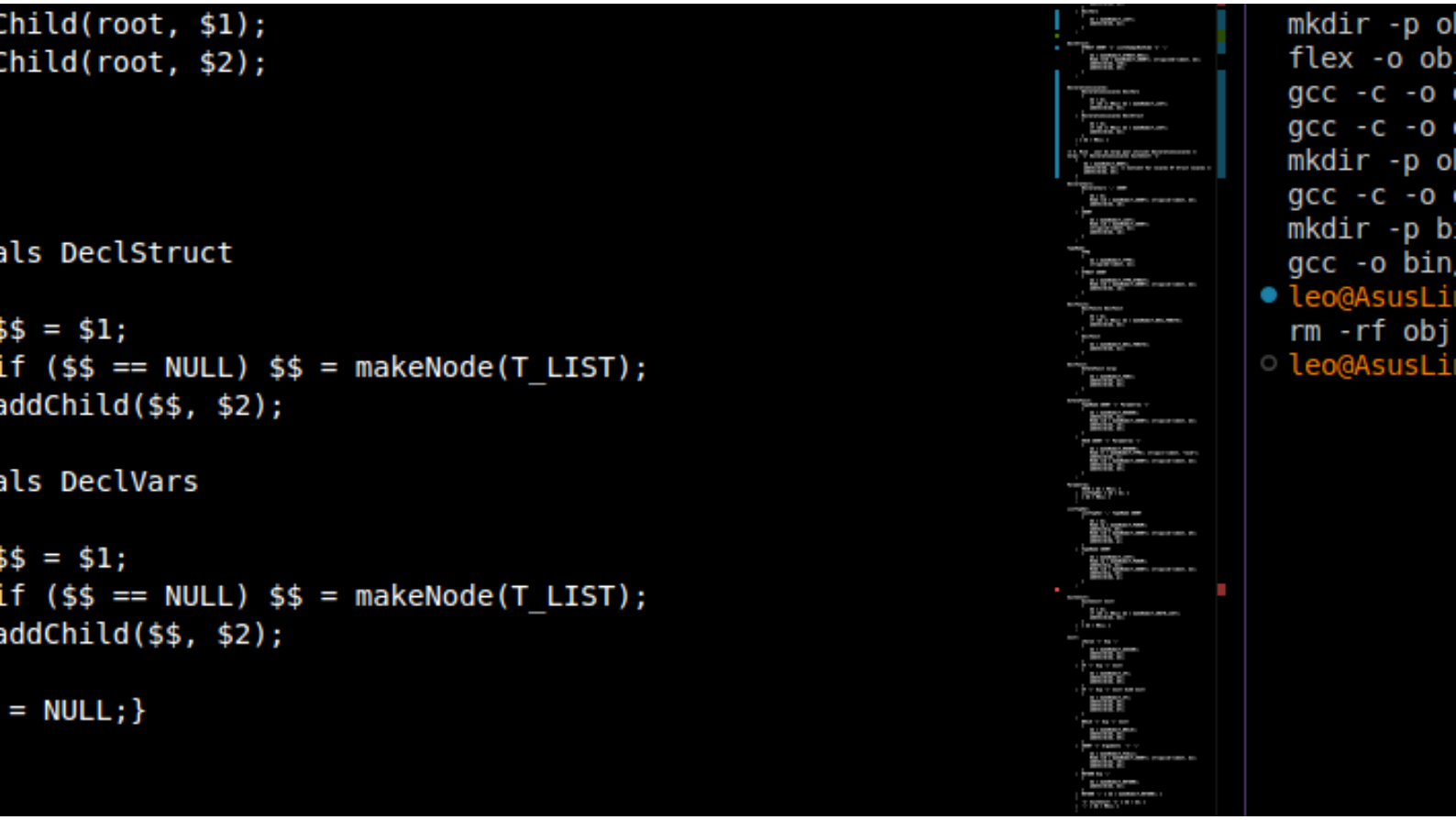
ANALYSE SYNTAXIQUE 2025 2026 – L3
LEO PIERRAT



1 – Manuel d’utilisation
2 – Description détaillée du programme
3 – Retours personnels

1 Manuel d’utilisation

Le programme peut être créé avec le makefile et make, l’exécutable s’appelle tpcas (pour TPC Analyseur Syntaxique)

```
$make
```

Une fois l’exécutable créé, vous pourrez l’utiliser de la façon suivante :
./tpcas [options] < monfichier.tpc

Le programme renverra 0 si le programme est valide, et 1 (ou plus) s’il ne l’est pas.

Les deux options sont :
-h (ou –help) : affiche un utilitaire comme celui de cette page
-t (ou –tree) : permet d’afficher l’arbre abstrait issu de bison, construit par l’analyse.

Par exemple, pour le programme prog1.tpc, avec la commande suivante, on obtient :

```
leo@AsusLinuxLeo:~/Documents/parsingL3 (main %) $./bin/tpcas -t < ./test/good/1.tpc

PROG
└── DECL_FONCTS
    └── FUNC
        ├── HEADER
        │   ├── TYPE: int
        │   └── IDENT: main
        └── BODY
            └── INSTR_LIST
                └── RETURN
                    └── NUM: 0
```

Avec le programme tpc suivant :

```
1  int main(void){
2      return 0;
3  }
```

```
leo@AsusLinuxLeo:~/Documents/parsingL3 (main *%) $./bin/tpcas -t < ./test/good/logik.tpc

PROG
└── DECL_FONCTS
    └── FUNC
        ├── HEADER
        │   ├── TYPE: int
        │   └── IDENT: main
        └── BODY
            ├── LIST
            │   └── DECL_VARS
            │       └── LIST
            │           ├── TYPE: int
            │           └── LIST
            │               ├── IDENT: x
            │               └── IDENT: y
            └── INSTR_LIST
                ├── ASSIGN
                │   ├── IDENT: x
                │   └── NUM: 1
                ├── ASSIGN
                │   ├── IDENT: y
                │   └── NUM: 0
                ├── IF
                │   ├── AND
                │   │   ├── EQ: ==
                │   │   │   ├── IDENT: x
                │   │   │   └── NUM: 1
                │   │   └── EQ: !=
                │   │       ├── IDENT: y
                │   │       └── NUM: 1
                │   └── INSTR_LIST
                │       └── IF
                │           ├── OR
                │           │   ├── ORDER: >=
                │           │   │   ├── IDENT: x
                │           │   │   └── IDENT: y
                │           │   └── NOT
                │           │       └── ORDER: <
                │           │           ├── IDENT: x
                │           │           └── NUM: 0
                │           └── INSTR_LIST
                │               └── RETURN
                │                   └── NUM: 1
                └── RETURN
                    └── NUM: 0
```

Voici le type d’arbres que l’on peut avoir avec des programmes plus longs.

2 – Description plus détaillée :

Le programme est construit avec flex et bison. Flex traîte le texte et envoit les lexèmes à Bison qui ensuit va vérifier que la syntaxe soit juste.
Parlons d'abord de Flex :

Le programme a un système de numérotation de ligne (note pour correction, je n'ai pas implémenté la numérotation de caractère)
J'ai donc un entier lineno, qui compte les lignes.

Ensuite on a les différentes règles, j'ai utilisé les préconditions de flex pour faire la gestion des commentaires sur plusieurs lignes (/* */). Lorsqu'on entre dans l'état COMM, on ignore tout sauf les \n pour garder le compteur de lignes actif.

Ensuite on a les mots de la syntaxe de tpc, les identifiants généraux, les opérations, etc. Les règles vont du plus spécifique au plus général.
Si à la toute fin on ne trouve aucun lexème avec le courant, on arrête le programme et on renvoit 1. Enfin si l'on atteint la fin du fichier, on renvoit 0 et on passe à Bison.

```
{IDENT} {
        strcpy(yylval.ident, yytext) ;
        return IDENT; }

[+-] {
        yylval.byte = yytext[0];
        return ADDSUB; }
[\/\*\%] {
        yylval.byte = yytext[0];
        return DIVSTAR; }

\'\\n\'         {yylval.character = '\n';
\'\\t\'         {yylval.character = '\t';
\'.\' {
    yylval.character = yytext[1];
    return CHARACTER;
}

[0-9]+ {
        yylval.num =atoi(yytext);
        return NUM; }
\&\& {
        return AND;}
\|\| {
        return OR;}
\=\=|\!\= {
        strcpy(yylval.comp, yytext) ;
        return EQ;}
[<>]|\<\=|\>\= {
        strcpy(yylval.comp, yytext) ;
        return ORDER;}
[!;,\(\)\{\}\[\]\.=] {
        return yytext[0];
}

{BLANCS} ;
\n {lineno++;}
. {exit(1);}
<*><<EOF>> {printf("\n"); return 0;}
```

Bison

Le programme bison est beaucoup plus long et beaucoup plus complexe, mais pour résumer, Bison va récupérer les lexèmes de Flex et vérifier que tout soit en ordre par rapport à la syntaxe de TPC. Je ne l'ai pas précisé plus tôt mais flex envoit à bison le type de lexème auquel on a affaire, et sa valeur, qui selon le type de valeur sera quelque chose de différent (attribut de yylval) et ces types sont définis dans l'union %union.

```
%union {
    int num;
    char ident[64];
    char byte;
    char character;
    char comp[4];
    Node *node;
}
```

Ensuite on définit les terminaux, les non terminaux et toutes les règles de la grammaire.

```
%token <num> NUM
%token <ident> IDENT TYPE
%token <character> CHARACTER
%token <byte> ADDSUB DIVSTAR
%token <comp> EQ ORDER
%token IF ELSE WHILE RETURN STRUCT VOID AND OR

%type <node> Prog DeclVars DeclFoncts DeclFonct EnTeteFonct Parametres ListTypVar
%type <node> Corps SuiteInstr Instr Exp TB FB M E T F Arguments ListExp
%type <node> DeclStruct TypeName LValue Declarateurs Globals
%type <node> ListChampsNonVide DeclarationsLocales
```

Les règles sont trop nombreuses pour être montrées, mais pour résumer leur établissement, à partir du fichier donné en consigne, on a du ajouter le traîtement des struct, et on a du implémenter la construction de l'arbre abstrait correspondant à l'analyse. Pour cela, à chaque règle, j'ai du ajouter des instructions en C qui appelent des fonctions définies dans le module tree faisant partie du projet. Le module tree permet de créer l'arbre avec ses différents nœuds, qui contiennent les labels des terminaux/non

terminaux de chaque nœud, et leur valeur, qui fut le principal défi d'implémentation.

Pour cela, dans le module tree, on doit faire en sorte que chaque nœud puisse contenir chaque type possible d'attribut. Lorsqu'on crée un nœud, on a donc plein de champs vides mais ce n'est pas grave car on sait quel champ est utile grâce au champ label qui lui n'est jamais vide.

```
typedef struct Node {
  label_t label;
  int lineno;
  struct Node *firstChild, *nextSibli
  char ident[64];
  int num;
  char byte;
  char character;
  char comp[4];
} Node;
```

```
switch(node->label) {
  /*on affiche la bonne donnée selon le type du label (les autres champs sont
  pas initialisés, pas besoin grâce à ca, on peut afficher les attributs)*/
    case T_IDENT:
    case T_TYPE:
        printf(": %s", node->ident);
        break;
    case T_NUM:
        printf(": %d", node->num);
        break;
    case T_CHARACTER:
        printf(": '%c'", node->character);
        break;
    case T_ADDSUB:
    case T_DIVSTAR:
        printf(": %c", node->byte);
        break;
    case T_EQ:
    case T_ORDER:
        printf(": %s", node->comp);
        break;
    default:
        break;
}
```

Exemple ici de l’axiome, qui comme on le voit crée le premier nœud de l’arbre. (root est déclaré au début du programme bison).

```
Prog:  Globals DeclFonctsS
    {
        root = makeNode(T_PROG);
        addChild(root, $1);
        addChild(root, $2);
    }
    ;
```

```
int main(int argc, char **argv) {
    int print = 0;
    if (argc > 1 && (strcmp(argv[1], "-t") == 0 || strcmp(argv[1], "--tree") == 0)) print = 1;
    if (argc > 1 && (strcmp(argv[1], "-h") == 0 || strcmp(argv[1], "--help") == 0)){
        printf("Utiliser l'analyseur syntaxique : \n\t ./tpcas [options] < fichier.tpc\n\n\tLes
        return 0;
    }
    if (yyparse() == 0) {
        /*si ya pas de problèmes !!!!*/
        if (print && root != NULL) {
            printTree(root);
        }
        return 0;
    }
    return 1 + nberror;
}
```

Enfin on a un main à la fin du programme bison qui trâite les arguments et renvoit le bon message d’erreur. ,